

MSU ROVER TEAM

ОТКРЫТЫЕ БИБЛИОТЕКИ ДЛЯ АВТОНОМНОЙ НАВИГАЦИИ ШЕСТИКОЛЕСНОГО ПРОТОТИПА МАРСОХОДА (РОВЕРА) ПО УМЕРЕННО ПЕРЕСЕЧЁННОЙ НЕЗНАКОМОЙ МЕСТНОСТИ С ВИЗУАЛЬНЫМ РАСПОЗНАВАНИЕМ ЦЕЛИ НАВИГАЦИИ.

СОГЛАСОВАНО:

Научный эксперт проекта, к.ф.-м.н.

 В.М. Буданов

05.12.2025

УТВЕРЖДАЮ:

Руководитель проекта

 А.А. Смирнов

05.12.2025

**Открытая библиотека распознавания указателей направления движения,
определения расстояния и направления к указателям.**

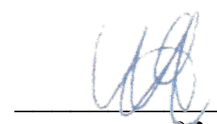
Руководство программиста.

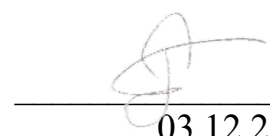
ЛИСТ УТВЕРЖДЕНИЯ

MSUROVERTEAM-CV-V1.0.0

(открытая библиотека в сети Интернет)

ИСПОЛНИТЕЛИ:

 Я.И. Шейпак
03.12.2025

 М.Д. Реппо
03.12.2025

MSU ROVER TEAM

УТВЕРЖДЕНО

**ОТКРЫТЫЕ БИБЛИОТЕКИ ДЛЯ АВТОНОМНОЙ НАВИГАЦИИ
ШЕСТИКОЛЕСНОГО ПРОТОТИПА МАРСОХОДА (РОВЕРА) ПО УМЕРЕННО
ПЕРЕСЕЧЁННОЙ НЕЗНАКОМОЙ МЕСТНОСТИ С ВИЗУАЛЬНЫМ
РАСПОЗНАВАНИЕМ ЦЕЛИ НАВИГАЦИИ.**

**Открытая библиотека распознавания указателей направления движения,
определения расстояния и направления к указателям.**

Руководство программиста.

MSUROVERTEAM-CV-V1.0.0

(открытая библиотека в сети Интернет)

Листов 26.

АННОТАЦИЯ.

Открытая библиотека распознавания указателей направления движения, определения расстояния и направления к указателям разработана в рамках проекта «Разработки открытых библиотек для автономной навигации шестиколесного прототипа марсохода (ровера) по умеренно пересечённой незнакомой местности с визуальным распознаванием цели навигации». Проект выполнен на средства выделенные «Фондом содействия развитию малых форм предприятий в научно-технической сфере» (Фонд содействия инновациям) по договору предоставления гранта № 64ГУКодИИС13-D7/102402 от 23 декабря 2024г.

Под полностью автономным режимом навигации (движения) в данном проекте понимается режим, при котором ровер с Аккермановой геометрией поворота самостоятельно, без команд оператора (человека), передвигается по умеренно пересечённой и незнакомой местности по указателям направления движения до указателя конечной цели, может выполнить заранее запрограммированные действия у каждого указателя и самостоятельно вернуться обратно к месту старта. При этом оператор может просматривать на своем мониторе видеоизображения и телеметрию, передаваемые с ровера.

В качестве указателя направления движения применяется знак (с белым фоном и с черной стрелкой), размером 300 x 200 мм, поднятый на высоту 100 - 150 мм над поверхностью. Размеры стрелки на знаке приведены в Приложении А к настоящему Руководству. В качестве указателя конечной цели используется оранжевый дорожный конус, с размерами соответствующим п.4.3.1.1 ГОСТ32758-2014. «Открытая библиотека распознавания указателей направления движения, определения расстояния и направления к указателям» предназначена для автоматического обнаружения/распознавания указателей направления движения и конечной цели, автоматического определения расстояния до них и передачи информации о детектированных навигационных объектах в модуль локализации для принятия решений о движении ровера.

«Открытая библиотека распознавания указателей направления движения, определения расстояния и направления к указателям» разработана на языке программирования Python 3, с использованием нейросетевой модели YOLOv8 (Ultralytics), для платформы ROS2 Humble (Robot Operating System 2 версии Humble).

СОДЕРЖАНИЕ.

АННОТАЦИЯ.	2
СОДЕРЖАНИЕ.	3
Общие сведения о программе.	4
Структура программы.	6
1. КЛАСС DetectionResult.	6
2. КЛАСС NavigationDetector.	7
3. ФУНКЦИЯ detect_arrow().	8
4. ФУНКЦИЯ detect_cone().	9
Интеграция с модулем локализации.	9
Пример использования библиотеки.	10
УСТАНОВКА БИБЛИОТЕКИ.	11
Пошаговая инструкция запуска примера.	12
Примеры вариантов использования библиотеки.	13
Обработка видеопотока с веб-камеры.	13
Пошаговая инструкция примера.	14
Обработка видеофайла с сохранением результатов/	15
Пошаговая инструкция примера.	16
Калибровка камеры для точного измерения расстояний	17
Пошаговая инструкция примера.	19
Интеграция с ROS 2 для управления роботом.	19
Пошаговая инструкция примера.	21
Пакетная обработка изображений	21
Пошаговая инструкция примера.	24
ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ.	24
Использование GPU	24
Оптимизация для встраиваемых систем	24
Мультипоточная обработка	25
ПРИЛОЖЕНИЕ А.	26

Общие сведения о программе.

«Открытая библиотека распознавания указателей направления движения, определения расстояния и направления к указателям» предназначена для автоматического обнаружения/распознавания указателей направления движения и конечной цели, автоматического определения расстояния до них и передачи информации о детектированных навигационных объектах в модуль локализации для принятия решений о движении шестиколесного прототипа марсохода (ровера) с Аккермановой геометрией поворота по умеренно пересечённой незнакомой местности.

В качестве указателя направления движения применяется знак (с белым фоном и с черной стрелкой), размером 300 x 200 мм, поднятый на высоту 100 - 150 мм над поверхностью. Размеры стрелки на знаке приведены в Приложении А к настоящему Руководству. В качестве указателя конечной цели используется оранжевый дорожный конус, с размерами соответствующим п.4.3.1.1 ГОСТ32758-2014.

ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ.

Платформа: ROS2 Humble (Robot Operating System 2 версии Humble).

Язык программирования: Python 3.

Нейросетевая модель: YOLOv8 (Ultralytics).

Зависимости:

- opencv-python (компьютерное зрение);
- numpy (численные вычисления);
- ultralytics (YOLO детекция);
- ROS2 (интеграция с роботом).

Измерения:

- калиброванное измерение расстояния методом кусочно-линейной интерполяции по таблице калибровки;
- измерение углов на основе модели камеры-обскуры.

Диапазон работы:

- расстояние: 1-10 метров (калиброванный диапазон);
- угол обзора: зависит от параметров камеры.

МИНИМАЛЬНЫЕ ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ:

- Операционная система: Ubuntu 22.046.
- Процессор: x86_64 или ARM64 (например, Nvidia Jetson).
- ОЗУ: 4 GB минимум, 8 GB рекомендуется.
- GPU: NVIDIA GPU с поддержкой CUDA 11.0+ (опционально, но рекомендуется).
- Дисковое пространство: 2 GB для библиотеки и моделей.
- Камера: USB/CSI камера с разрешением минимум 640x480.

Рекомендуемые требования (тестируемая конфигурация)

- Платформа: Nvidia Jetson Orin NX Super.
- Камера: Lucid Triton TRI016S-CC или аналогичная промышленная камера.
- ОЗУ: 16 GB.
- Хранилище: NVMe SSD 256 GB.

Программные требования

- Операционная система.
 - Ubuntu 20.04/22.04 LTS.
 - Windows:** Windows 10/11 (с WSL2 для полной функциональности).
 - Jetson:** JetPack 5.0+ (для Nvidia Jetson).
- Python и основные зависимости.
 - Python 3.8-3.11
 - pip >= 21.0
- Обязательные Python библиотеки
 - ultralytics>=8.0.0 # YOLO v8 framework
 - torch>=1.10.0 # PyTorch (с CUDA если есть GPU)
 - torchvision>=0.11.0 # Computer vision models
 - opencv-python>=4.5.0 # OpenCV для обработки изображений
 - numpy>=1.20.0 # Численные вычисления
- Опциональные зависимости (для ROS интеграции).
 - rclpy # ROS 2 Python клиент (для nav_simple.py)
 - geometry_msgs # ROS 2 сообщения геометрии
 - sensor_msgs # ROS 2 сенсорные сообщения

Модель нейронной сети

- Файл весов: weights/best.pt (включен в репозиторий)
- Архитектура: YOLOv8n (nano)
- Классы: 2 (arrow - стрелка, cone - конус)
- Размер модели: ~6 MB

-СПИСОК ОБЪЕКТОВ ДЛЯ ДОКУМЕНТИРОВАНИЯ.

1. DetectionResult - класс данных результата распознавания.
2. NavigationDetector - основной класс детектора.
3. detect_arrow() - функция распознавания стрелок (left/right).
4. detect_cone() - функция распознавания дорожных конусов.

Дополнительные служебные методы (не требуют отдельного документирования):

- __init__() - конструктор класса;
- detect_all() - комплексная детекция.

Структура программы.

Модуль: eureka_nav_lib.py

Публичные объекты библиотеки:

1. Класс DetectionResult (структура данных);
2. Класс NavigationDetector (основной класс детектора);
3. Функция detect_arrow() (распознавание стрелок);
4. Функция detect_cone() (распознавание дорожных конусов).

1. КЛАСС DetectionResult.

Описание: структура данных для хранения результата распознавания навигационного объекта (стрелки или дорожного конуса).

Тип: dataclass.

Поля (атрибуты).

- object_type: str
Тип обнаруженного объекта
Значения: "arrow" (стрелка), "cone" (дорожный конус)
- direction: str
Направление объекта для навигации
Значения: "left" (налево), "right" (направо), "none" (нет направления)
- distance_m: float
Расстояние до объекта в метрах (калиброванное измерение)
- angle_deg: float
Угол отклонения объекта от центра камеры в градусах
Отрицательные значения - справа, положительные - слева
- confidence: float
Уверенность детекции в диапазоне [0.0, 1.0]
- bbox: tuple[int, int, int, int]
Ограничивающий прямоугольник детекции в формате (x1, y1, x2, y2)

Назначение: передача информации о детектированных навигационных объектах в модуль локализации для принятия решений о движении ровера.

2. КЛАСС NavigationDetector.

Описание: основной класс детектора навигационных объектов для автономного марсохода.

Обеспечивает распознавание стрелок и дорожных конусов с калиброванным измерением расстояния и угла.

Методы инициализации:

- `__init__(weights_path: str)`
Инициализация детектора с загрузкой модели машинного обучения.
Входные параметры:
 - `weights_path`: Путь к файлу весов YOLO модели (.pt файл)
Возвращаемое значение: нет

Публичные методы:

- `detect_arrow(image: np.ndarray) -> List[DetectionResult]`
Назначение: распознавание стрелок с определением направления движения (левая или правая стрелка).
Входные параметры:
 - `image`: Изображение в формате BGR (numpy array, цветное изображение)
Возвращаемое значение: список объектов `DetectionResult`, содержащих информацию о всех обнаруженных стрелках. Поле `direction` содержит "left" или "right".
Применение: используется для определения направления движения ровера по стрелкам на местности.
- `detect_cone(image: np.ndarray) -> List[DetectionResult]`
Назначение: распознавание дорожных конусов для определения целей навигации.
Входные параметры:
 - `image`: Изображение в формате BGR (numpy array, цветное изображение)
Возвращаемое значение: список объектов `DetectionResult`, содержащих информацию о всех обнаруженных конусах. Поле `direction` для конусов всегда "none".
Применение: используется для обнаружения целевых точек навигации (конусов) на местности.
- `detect_all(image: np.ndarray) -> List[DetectionResult]`
Назначение: распознавание всех навигационных объектов одновременно (стрелки и конусы).
Входные параметры:
 - `image`: Изображение в формате BGR (numpy array, цветное изображение)
Возвращаемое значение: список всех обнаруженных объектов `DetectionResult` (стрелки и конусы).
Применение: комплексный анализ окружения для навигации ровера.

3. ФУНКЦИЯ `detect_arrow()`.

- Полное имя: `NavigationDetector.detect_arrow()`
- Назначение: функция распознавания стрелок (левых и правых) на изображении с камеры ровера для определения направления движения.
- Входные данные:
 - `image`: Изображение с камеры ровера (numpy array, BGR формат).
- Выходные данные:
 - список результатов детекции (`List[DetectionResult]`);
 - каждый результат содержит:
 - тип объекта: "arrow",
 - направление: "left" (левая стрелка) или "right" (правая стрелка),
 - калиброванное расстояние в метрах,
 - угол отклонения в градусах,
 - уверенность детекции (0.0-1.0).
- Алгоритм:
 - детекция стрелок нейросетью YOLO;
 - анализ формы стрелки методом PCA с мажоритарным голосованием:
 - распределение массы пикселей,
 - градиент ширины контура,
 - остроконечность (поиск самого острого угла);
 - калиброванное измерение расстояния по таблице калибровки;
 - вычисление угла относительно центра камеры.
- Применение: основная функция для визуального распознавания направления движения ровера по стрелкам на местности.

4. ФУНКЦИЯ `detect_cone()`.

- Полное имя: `NavigationDetector.detect_cone()`
- Назначение: функция распознавания дорожных конусов на изображении с камеры ровера для определения целей навигации.
- Входные данные:
 - `image`: Изображение с камеры ровера (numpy array, BGR формат).
- Выходные данные:
 - список результатов детекции (`List[DetectionResult]`);
 - каждый результат содержит:
 - тип объекта: "cone",
 - направление: "none" (для конусов не определяется),
 - калиброванное расстояние в метрах,
 - угол отклонения в градусах,
 - уверенность детекции (0.0-1.0).
- Алгоритм:
 - детекция конусов нейросетью YOLO;
 - фильтрация детекций по порогу уверенности;
 - подавление избыточных детекций (Non-Maximum Suppression);
 - калиброванное измерение расстояния по таблице калибровки;
 - вычисление угла относительно центра камеры.
- Применение: функция для обнаружения целевых точек навигации (конусов) на местности, к которым должен двигаться ровер.

Интеграция с модулем локализации.

Библиотека предоставляет набор объектов для передачи информации о навигационных объектах в модуль локализации «Библиотеки автономной навигации по распознанным указателям движения».

1. `DetectionResult` содержит все необходимые данные для локализации:
 - a. Тип объекта (стрелка/конус);
 - b. Направление движения (left/right);
 - c. Расстояние до объекта (метры);
 - d. Угловое положение (градусы);
 - e. Уверенность детекции.
2. `NavigationDetector.detect_arrow()` - основная функция распознавания стрелок с выходом "левая стрелка" или "правая стрелка".
3. `NavigationDetector.detect_cone()` - функция распознавания дорожных конусов как целей навигации.

Пример использования библиотеки.

Python код:

```
#!/usr/bin/env python3
#Базовый пример использования библиотеки MSUROVERTEAM-CV
#Детекция стрелок и конусов на изображении

import cv2
import sys
from pathlib import Path

# Добавляем путь к библиотеке
sys.path.append(str(Path(__file__).parent))

from eureka_nav_lib import NavigationDetector

def main():
    # Шаг 1: Инициализация детектора
    print("Инициализация детектора...")
    detector = NavigationDetector(
        weights_path="weights/best.pt", # Путь к файлу весов
        device=None # Автоматический выбор устройства (GPU если доступно)
    )

    # Шаг 2: Загрузка изображения
    print("Загрузка изображения...")
    image_path = "test_image.jpg" # Замените на путь к вашему изображению
    image = cv2.imread(image_path)

    if image is None:
        print(f"Ошибка: не удалось загрузить изображение {image_path}")
        return

    # Шаг 3: Детекция всех объектов
    print("Выполнение детекции...")
    detections = detector.detect_all(image)

    # Шаг 4: Вывод результатов
    print(f"\nОбнаружено объектов: {len(detections)}")
    print("-" * 60)

    for i, det in enumerate(detections, 1):
        print(f"Объект #{i}:")
        print(f"  Тип: {det.object_type}")
        print(f"  Направление: {det.direction}")
        print(f"  Расстояние: {det.distance_m:.2f} м")
        print(f"  Угол: {det.angle_deg:.1f}°")
```

```
print(f" Уверенность: {det.confidence:.2%}")
print(f" Координаты: {det.bbox}")
print()

# Шаг 5: Визуализация результатов
for det in detections:
    x1, y1, x2, y2 = det.bbox

    # Выбор цвета в зависимости от типа
    color = (0, 255, 0) if det.object_type == "arrow" else (0, 165, 255)

    # Рисуем прямоугольник
    cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)

    # Подготовка текста
    label = f"{det.object_type}"
    if det.direction != "none":
        label += f" {det.direction}"
    label += f" {det.distance_m:.1f}m"

    # Рисуем текст
    cv2.putText(image, label, (x1, y1-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

# Сохранение результата
output_path = "result.jpg"
cv2.imwrite(output_path, image)
print(f"Результат сохранен в {output_path}")

# Показ результата (если есть дисплей)
cv2.imshow("Detection Result", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

УСТАНОВКА БИБЛИОТЕКИ.

- Клонирование репозитория

```
git clone https://github.com/KodII-rover/MSUROVERTEAM-CV.git
cd MSUROVERTEAM-CV
```

- Создание виртуального окружения (рекомендуется)

```
python3 -m venv venv
source venv/bin/activate # Linux/Mac
# или
venv\Scripts\activate   # Windows
```

- Установка зависимостей

- Вариант 1: Минимальная установка (без GPU)

```
pip install ultralytics opencv-python numpy
```

- Вариант 2: Установка с поддержкой CUDA (NVIDIA GPU)

Установка PyTorch с CUDA (проверьте версию CUDA командой `nvidia-smi`)

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118 # для CUDA 11.8
```

Установка остальных зависимостей

```
pip install ultralytics opencv-python numpy
```

- Установка для Jetson

На Jetson платформах PyTorch обычно предустановлен в JetPack

```
pip install ultralytics opencv-python numpy
```

- Проверка установки

```
python3 -c "import torch; print(f'PyTorch: {torch.__version__}')"
python3 -c "import cv2; print(f'OpenCV: {cv2.__version__}')"
python3 -c "from ultralytics import YOLO; print('YOLO загружен успешно')"
```

Пошаговая инструкция запуска примера.

1. Подготовка тестового изображения:

```
# Поместите изображение со стрелкой или конусом
# в корневую директорию репозитория (MSUROVERTEAM-CV/)
# Назовите его test_image.jpg
```

2. Запуск скрипта:

```
python3 example_basic.py
```

3. Ожидаемый вывод:

Инициализация детектора...
Загрузка изображения...
Выполнение детекции...

Обнаружено объектов: 2

Объект #1:
Тип: arrow
Направление: left
Расстояние: 3.50 м
Угол: -15.2°
Уверенность: 92.30%
Координаты: (120, 150, 250, 280)

Примеры вариантов использования библиотеки.

Обработка видеопотока с веб-камеры.

Создайте файл `example\_webcam.py`:

```
#!/usr/bin/env python3
# Пример обработки видеопотока с веб-камеры в реальном времени

import cv2
import sys
from pathlib import Path

sys.path.append(str(Path(__file__).parent))
from eureka_nav_lib import NavigationDetector

def process_webcam():
    # Инициализация детектора
    detector = NavigationDetector("weights/best.pt")

    # Открытие веб-камеры (0 - камера по умолчанию)
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Ошибка: не удалось открыть камеру")
        return

    print("Нажмите 'q' для выхода")

    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

```
# Детекция объектов
detections = detector.detect_all(frame)

# Визуализация
for det in detections:
    x1, y1, x2, y2 = det.bbox
    color = (0, 255, 0) if det.object_type == "arrow" else (0, 165, 255)

    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

    # Текст с информацией
    info = f"{det.object_type}"
    if det.direction != "none":
        info += f" {det.direction}"
    info += f" {det.distance_m:.1f}m"

    cv2.putText(frame, info, (x1, y1-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

# Показ FPS
cv2.putText(frame, f"Objects: {len(detections)}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

cv2.imshow("Navigation Detection", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

if __name__ == "__main__":
    process_webcam()
```

Пошаговая инструкция примера.

python3 example_webcam.py

Обработка видеофайла с сохранением результатов/

Создайте файл `example\_video\_processing.py`:

```
#!/usr/bin/env python3
#Обработка видеофайла с сохранением аннотированного видео

import cv2
import sys
from pathlib import Path
import time

sys.path.append(str(Path(__file__).parent))
from eureka_nav_lib import NavigationDetector

def process_video(input_path, output_path):
    # Инициализация
    detector = NavigationDetector("weights/best.pt")

    # Открытие видео
    cap = cv2.VideoCapture(input_path)
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Настройка записи
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(output_path, fourcc, fps, (width, height))

    print(f"Обработка видео: {total_frames} кадров")
    frame_count = 0
    start_time = time.time()

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        frame_count += 1

        # Детекция
        detections = detector.detect_all(frame)

        # Отрисовка результатов
        for det in detections:
            x1, y1, x2, y2 = det.bbox
            color = (0, 255, 0) if det.object_type == "arrow" else (0, 165, 255)

            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
```



```
label = f"{det.object_type}"
if det.direction != "none":
    label += f" {det.direction}"
label += f" {det.distance_m:.1f}m | {det.angle_deg:.0f}°"

cv2.putText(frame, label, (x1, y1-10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

# Информационная панель
cv2.rectangle(frame, (0, 0), (width, 40), (0, 0, 0), -1)
info = f"Frame: {frame_count}/{total_frames} | Objects: {len(detections)}"
cv2.putText(frame, info, (10, 25),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

# Запись кадра
out.write(frame)

# Прогресс
if frame_count % 30 == 0:
    elapsed = time.time() - start_time
    fps_processing = frame_count / elapsed
    eta = (total_frames - frame_count) / fps_processing
    print(f"Прогресс: {frame_count}/{total_frames} "
          f"({frame_count/total_frames*100:.1f}%) "
          f"FPS: {fps_processing:.1f} ETA: {eta:.0f}s")

# Освобождение ресурсов
cap.release()
out.release()
cv2.destroyAllWindows()

elapsed = time.time() - start_time
print(f"\nОбработка завершена за {elapsed:.1f} секунд")
print(f"Результат сохранен в {output_path}")

if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description="Обработка видеофайла")
    parser.add_argument("input", help="Путь к входному видео")
    parser.add_argument("-o", "--output", default="output.mp4",
                        help="Путь к выходному видео")

    args = parser.parse_args()
    process_video(args.input, args.output)
```

Пошаговая инструкция примера.

```
python3 example_video_processing.py input_video.mp4 -o result_video.mp4
```

Калибровка камеры для точного измерения расстояний

Создайте файл `example\_calibration.py`:

```
#!/usr/bin/env python3
#Калибровка камеры для точного измерения расстояний
#Поместите стрелку на известных расстояниях и измерьте размеры в пикселях

import cv2
import sys
from pathlib import Path
import json

sys.path.append(str(Path(__file__).parent))
from eureka_nav_lib import NavigationDetector

def calibrate_camera():
    detector = NavigationDetector("weights/best.pt")
    cap = cv2.VideoCapture(0)

    calibration_data = {
        "arrows": [],
        "cones": []
    }

    print("КАЛИБРОВКА КАМЕРЫ")
    print("-" * 40)
    print("Инструкции:")
    print("1. Поместите объект на известное расстояние")
    print("2. Нажмите ПРОБЕЛ для захвата")
    print("3. Введите расстояние в метрах")
    print("4. Повторите для разных расстояний (1-10м)")
    print("5. Нажмите 'q' для завершения")
    print("-" * 40)

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # Детекция
        detections = detector.detect_all(frame)

        # Визуализация
        for det in detections:
            x1, y1, x2, y2 = det.bbox
            width = x2 - x1
            height = y2 - y1

            color = (0, 255, 0) if det.object_type == "arrow" else (0, 165, 255)
```

```
cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

# Показываем размеры в пикселях
info = f"{det.object_type} | W:{width}px H:{height}px"
cv2.putText(frame, info, (x1, y1-10),
             cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

cv2.imshow("Calibration", frame)

key = cv2.waitKey(1) & 0xFF

if key == ord(' ') and len(detections) > 0:
    # Захват калибровочной точки
    det = detections[0] # Берем первый обнаруженный объект
    x1, y1, x2, y2 = det.bbox
    width = x2 - x1
    height = y2 - y1

    # Запрос расстояния у пользователя
    cv2.imwrite("calibration_capture.jpg", frame)
    distance = float(input(f"Введите расстояние до {det.object_type} (м): "))

    # Сохранение данных
    calib_point = {
        "distance_m": distance,
        "width_px": width,
        "height_px": height
    }

    if det.object_type == "arrow":
        calibration_data["arrows"].append(calib_point)
    else:
        calibration_data["cones"].append(calib_point)

    print(f"Сохранено: {distance}м -> {width}x{height}px")
    print(f"Точек для стрелок: {len(calibration_data['arrows'])}")
    print(f"Точек для конусов: {len(calibration_data['cones'])}")

elif key == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

# Сохранение калибровочных данных
with open("calibration_data.json", "w") as f:
    json.dump(calibration_data, f, indent=2)

print("\nКалибровка завершена!")
```

```
print("Данные сохранены в calibration_data.json")

# Генерация Python кода для calibration_config.py
print("\nДобавьте следующие данные в calibration_config.py:")
print("\nCALIBRATION_TABLE_ARROWS = [")
for point in sorted(calibration_data["arrows"], key=lambda x: x["distance_m"]):
    print(f"    ({point['distance_m']}, {point['width_px']}, {point['height_px']}),")
print("]")

print("\nCALIBRATION_TABLE_CONES = [")
for point in sorted(calibration_data["cones"], key=lambda x: x["distance_m"]):
    print(f"    ({point['distance_m']}, {point['width_px']}, {point['height_px']}),")
print("]")

if __name__ == "__main__":
    calibrate_camera()
```

Пошаговая инструкция примера.

python3 example_calibration.py

Интеграция с ROS 2 для управления роботом

Создайте файл `example\_ros\_integration.py`:

```
#!/usr/bin/env python3
#Интеграция с ROS 2 для передачи данных детекции в систему управления

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, JointState
from cv_bridge import CvBridge
import cv2
import sys
from pathlib import Path

sys.path.append(str(Path(__file__).parent))
from eureka_nav_lib import NavigationDetector

class NavigationDetectorNode(Node):
    def __init__(self):
        super().__init__('navigation_detector')

        # Инициализация детектора
        self.detector = NavigationDetector("weights/best.pt")

        # ROS интерфейсы
        self.bridge = CvBridge()

        # Подписка на изображения с камеры
        self.image_sub = self.create_subscription(
```

```
        Image,
        '/camera/image_raw',
        self.image_callback,
        10
    )

    # Публикация результатов детекции
    self.detection_pub = self.create_publisher(
        JointState, # Используем JointState для совместимости с nav_simple
        'arrow_detection',
        10
    )

    self.get_logger().info('Navigation Detector Node запущен')

def image_callback(self, msg):
    # Конвертация ROS Image в OpenCV
    cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")

    # Детекция
    detections = self.detector.detect_all(cv_image)

    # Подготовка сообщения с результатами
    detection_msg = JointState()
    detection_msg.header.stamp = self.get_clock().now().to_msg()

    for det in detections:
        detection_msg.name.append(det.direction)
        detection_msg.position.append(det.distance_m)
        detection_msg.velocity.append(det.angle_deg)
        detection_msg.effort.append(det.confidence)

    # Публикация результатов
    self.detection_pub.publish(detection_msg)

    # Логирование
    if len(detections) > 0:
        self.get_logger().info(
            f'Обнаружено объектов: {len(detections)}'
        )

def main(args=None):
    rclpy.init(args=args)
    node = NavigationDetectorNode()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
```

```
finally:  
    node.destroy_node()  
    rclpy.shutdown()  
  
if __name__ == '__main__':  
    main()
```

Пошаговая инструкция примера.

```
# Терминал 1: Запуск камеры  
ros2 run usb_cam usb_cam_node_exe  
  
# Терминал 2: Запуск детектора  
python3 example_ros_integration.py  
  
# Терминал 3: Запуск навигации (опционально)  
python3 nav_simple.py python3 example_calibration.py
```

Пакетная обработка изображений

Создайте файл `example\_batch\_processing.py`:

```
#!/usr/bin/env python3  
#Пакетная обработка множества изображений с генерацией отчета  
  
import cv2  
import sys  
from pathlib import Path  
import json  
from datetime import datetime  
  
sys.path.append(str(Path(__file__).parent))  
from eureka_nav_lib import NavigationDetector  
  
def batch_process(input_folder, output_folder):  
    # Создание выходной директории  
    output_path = Path(output_folder)  
    output_path.mkdir(parents=True, exist_ok=True)  
  
    # Инициализация детектора  
    detector = NavigationDetector("weights/best.pt")  
  
    # Получение списка изображений  
    image_extensions = ['.jpg', '.jpeg', '.png', '.bmp']  
    image_files = []  
    for ext in image_extensions:  
        image_files.extend(Path(input_folder).glob(f'*{ext}'))  
        image_files.extend(Path(input_folder).glob(f'*{ext.upper()}'))  
  
    print(f"Найдено изображений: {len(image_files)}")
```

```
# Результаты для отчета
report = {
    "timestamp": datetime.now().isoformat(),
    "total_images": len(image_files),
    "processed_images": 0,
    "total_detections": 0,
    "arrows_detected": 0,
    "cones_detected": 0,
    "results": []
}

# Обработка каждого изображения
for img_path in image_files:
    print(f"Обработка: {img_path.name}")

    # Загрузка изображения
    image = cv2.imread(str(img_path))
    if image is None:
        print(f" Ошибка загрузки, пропуск")
        continue

    # Детекция
    detections = detector.detect_all(image)

    # Статистика
    arrows = [d for d in detections if d.object_type == "arrow"]
    cones = [d for d in detections if d.object_type == "cone"]

    # Сохранение результата
    img_result = {
        "filename": img_path.name,
        "detections_count": len(detections),
        "arrows": len(arrows),
        "cones": len(cones),
        "details": []
    }

    # Визуализация
    for det in detections:
        x1, y1, x2, y2 = det.bbox
        color = (0, 255, 0) if det.object_type == "arrow" else (0, 165, 255)

        cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)

        label = f"{det.object_type}"
        if det.direction != "none":
            label += f" {det.direction}"
        label += f" {det.distance_m:.1f}m"
```

```
cv2.putText(image, label, (x1, y1-10),
             cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

# Добавление в отчет
img_result["details"].append({
    "type": det.object_type,
    "direction": det.direction,
    "distance_m": round(det.distance_m, 2),
    "angle_deg": round(det.angle_deg, 1),
    "confidence": round(det.confidence, 3)
})

# Сохранение аннотированного изображения
output_file = output_path / f"annotated_{img_path.name}"
cv2.imwrite(str(output_file), image)

# Обновление отчета
report["processed_images"] += 1
report["total_detections"] += len(detections)
report["arrows_detected"] += len(arrows)
report["cones_detected"] += len(cones)
report["results"].append(img_result)

print(f" Обнаружено: {len(detections)} объектов")

# Сохранение отчета
report_path = output_path / "detection_report.json"
with open(report_path, "w") as f:
    json.dump(report, f, indent=2, ensure_ascii=False)

# Вывод итоговой статистики
print("\n" + "="*50)
print("ИТОГОВАЯ СТАТИСТИКА")
print("="*50)
print(f"Обработано изображений: {report['processed_images']}")
print(f"Всего обнаружено объектов: {report['total_detections']}")
print(f" - Стрелок: {report['arrows_detected']}")
print(f" - Конусов: {report['cones_detected']}")
print(f"\nОтчет сохранен в: {report_path}")
print(f"Аннотированные изображения в: {output_path}")

if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(
        description="Пакетная обработка изображений"
    )
    parser.add_argument("input", help="Папка с входными изображениями")
```



```
parser.add_argument("-o", "--output", default="batch_results",
                    help="Папка для результатов")

args = parser.parse_args()
batch_process(args.input, args.output)
```

Пошаговая инструкция примера.

```
# Обработка всех изображений в папке
python3 example_batch_processing.py ./images -o ./results

# Результаты будут в папке results/:
# - annotated_*.jpg - изображения с разметкой
# - detection_report.json - подробный отчет
```

ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ.

Использование GPU

```
# Явное указание GPU
detector = NavigationDetector("weights/best.pt", device="cuda")

# Проверка использования GPU
import torch
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"VRAM: {torch.cuda.get_device_properties(0).total_memory / 1024**3:.1f} GB")
```

Оптимизация для встраиваемых систем

```
# Для Jetson - использование TensorRT
detector = NavigationDetector("weights/best.pt")
# YOLO автоматически использует TensorRT если доступен

# Уменьшение разрешения для ускорения
image_small = cv2.resize(image, (640, 480))
detections = detector.detect_all(image_small)

# Масштабирование координат обратно
scale_x = original_width / 640
scale_y = original_height / 480
for det in detections:
    x1, y1, x2, y2 = det.bbox
    det.bbox = (int(x1*scale_x), int(y1*scale_y),
               int(x2*scale_x), int(y2*scale_y))
```

Мультипоточная обработка

```
import concurrent.futures
from threading import Lock

detector = NavigationDetector("weights/best.pt")
results_lock = Lock()
results = []

def process_image(img_path):
    image = cv2.imread(str(img_path))
    detections = detector.detect_all(image)

    with results_lock:
        results.append({
            "file": img_path.name,
            "detections": len(detections)
        })

# Параллельная обработка
with concurrent.futures.ThreadPoolExecutor(max_workers=4) as executor:
    executor.map(process_image, image_files)
```

ПРИЛОЖЕНИЕ А.

Указатель направления движения.

